

Создание подписей к изображениям с помощью трансформеров

В этом блокноте необходимо реализовать основные части декодера трансформера, который создает подписи к изображениям.

Ввод [1]:

```
1 # Необходимые установки
2 import time, os, json
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 from dlc.v.gradient_check import eval_numerical_gradient, eval_numerical
7 from dlc.v.transformer_layers import *
8 from dlc.v.captioning_solver_transformer import CaptioningSolverTransform
9 from dlc.v.classifiers.transformer import CaptioningTransformer
10 from dlc.v.coco_utils import load_coco_data, sample_coco_minibatch, deco
11 from dlc.v.image_utils import image_from_url
12
13 %matplotlib inline
14 plt.rcParams['figure.figsize'] = (10.0, 8.0) # Размер графика по умолча
15 plt.rcParams['image.interpolation'] = 'nearest'
16 plt.rcParams['image.cmap'] = 'gray'
17
18 %load_ext autoreload
19 %autoreload 2
20
21 def rel_error(x, y):
22     """ возвращает относительную ошибку """
```

Установка h5py

Набор данных COCO, который мы будем использовать, хранится в формате HDF5. Чтобы загружать файлы HDF5, нужно установить пакет h5py Python. Из командной строки запустите:

```
pip install h5py
```

Если вы получили ошибку доступа, то может потребоваться выполнить команду от имени пользователя root:

```
sudo pip install h5py
```

Вы также можете запускать команды прямо из Jupyter notebook, поставив перед командой префикс "!":

Выполните команду, приведенную ниже в ячейке

Возможно при установке пакета h5py Вы получите сообщения о том, что элементы пакета уже установлены.:

```
Requirement already satisfied: h5py in c:\users\hp\anaconda3\envs\python36\lib\site-packages (3.1.0)
```

```
Requirement already satisfied: numpy>=1.12 in c:
```

```
\users\hp\anaconda3\envs\python36\lib\site-packages (from h5py) (1.19.2)
```

```
Requirement already satisfied: cached-property in c:
```

```
\users\hp\anaconda3\envs\python36\lib\site-packages (from h5py) (1.5.2)
```

Ввод [3]:

```
Requirement already satisfied: h5py in c:\users\hp\anaconda3\envs\python36\lib\site-packages (3.1.0)
Requirement already satisfied: numpy>=1.12 in c:\users\hp\anaconda3\envs\python36\lib\site-packages (from h5py) (1.19.2)
Requirement already satisfied: cached-property in c:\users\hp\anaconda3\envs\python36\lib\site-packages (from h5py) (1.5.2)
```

Датасет Microsoft COCO

В этом задании мы будем использовать набор данных Microsoft COCO (<http://mscoco.org/> (<http://mscoco.org/>)) 2014 года, который стал стандартным испытательным набором для изображений с субтитрами. Набор данных состоит из 80 000 обучающих изображений и 40 000 проверочных изображений, каждое из которых снабжено 5 подписями.

Объем данных COCO составляет ~ 1 ГБ. Он должен быть загружен в каталог `d1cv/datasets/coco_captioning` в предварительно обработанном виде (см.ниже).

Для этого надо скачать файл `train_val_VGG_pcah5.zip` и разархивировать его в указанный каталог. Если у Вас возникнут затруднения с загрузкой набора данных COCO, обратитесь к преподавателю, чтобы скопировать набор данных и разместить его в указанном каталоге вручную.

Для всех изображений набора данных COCO были извлечены признаки на уровне fc7 для сети VGG-16, предварительно обученной на данных ImageNet. Чтобы уменьшить время обработки и снизить требования к памяти, размерность признаков была сокращена с 4096 до 512 путем сжатия с использованием метода главных компонент; эти признаки были сохранены в файлах `train2014_vgg16_fc7_pca.h5` и `val2014_vgg16_fc7_pca.h5`, которые и находятся в архиве `train_val_VGG_pcah5.zip`.

Исходные изображения занимают в памяти много места (около 19 ГБ), поэтому они не были включены в архив. Все изображения взяты из системы Flickr, а URL-адреса обучающих и проверочных изображений хранятся в файлах `train2014_urls.txt` и `val2014_urls.txt`, соответственно. Это позволяет загружать изображения на лету для визуализации. Поскольку изображения загружаются на лету, **вы должны быть подключены к Интернету для просмотра изображений.**

Работа со строками неэффективна, поэтому мы будем работать с кодированной версией подписи к изображению. Каждому слову присваивается целочисленный идентификатор, что позволяет нам представлять подписи изображений последовательностью целых чисел. Таблица соответствия между целочисленными идентификаторами и словами находится в файле `coco2014_vocab.json`. Вы можете использовать функцию `decode_captions` из файла `d1cv/coco_utils.py` для преобразования массивов целочисленных идентификаторов обратно в строки.

Есть пара специальных токенов (лексем), которые добавлены в словарь. Это специальный токен `<START>` и токен `<END>`, которые добавляются в начало и конец каждого заголовка изображения, соответственно. Редкие слова заменяются

специальным токеном <UNK> ("unknown"- «неизвестно»). Кроме того, поскольку при обучении используются мини-блоки, содержащие заголовки различной длины, то короткие подписи дополняются специальным токеном <NULL> после токена <END> и для <NULL> токенов не вычисляются потери или градиенты. Так как обработка специальных токенов создает некоторые сложности, то Вам предлагается готовое решение, которое избавит Вас от необходимости писать соответствующий код.

Вы можете загрузить все данные MS-COCO (заголовки, функции, URL-адреса и словарь), используя функцию `load_coco_data` из файла `dlcv/ coco_utils.py` . Запустите ниже ячейку блокнота, чтобы сделать это:

Ввод [2]:

```
1 # Загрузка данных COCO с диска; функция возвращает словарь
2 # В этом блокноте будем работать только с сокращенной размерностью при
3 data = load_coco_data(pca_features=True)
4
5 # Печать названий ключей, типов значений словаря данных, размеров и тип
6 for k, v in data.items():
7     if type(v) == np.ndarray:
8         print(k, type(v), v.shape, v.dtype)
9     else:
```

```
base dir C:\Users\HP\DL_lab\2026_transformer_ решение\dlcv\datasets/coco_
captioning
train_captions <class 'numpy.ndarray'> (400135, 17) int32
train_image_idxes <class 'numpy.ndarray'> (400135,) int32
val_captions <class 'numpy.ndarray'> (195954, 17) int32
val_image_idxes <class 'numpy.ndarray'> (195954,) int32
train_features <class 'numpy.ndarray'> (82783, 512) float32
val_features <class 'numpy.ndarray'> (40504, 512) float32
idx_to_word <class 'list'> 1004
word_to_idx <class 'dict'> 1004
train_urls <class 'numpy.ndarray'> (82783,) <U63
val_urls <class 'numpy.ndarray'> (40504,) <U63
```

Трансформер

Как вы уже поняли, RNN невероятно мощные, но часто они медленно обучаемы. Кроме того, RNN с трудом кодируют длинные зависимости в элементах данных, которые находятся друг от друга на больших расстояниях (хотя LSTM — несколько смягчают эту проблему).

В 2017 году Васвани и др. представили архитектуру "Трансформер" в статье ["Attention Is All You Need"](https://arxiv.org/abs/1706.03762) (<https://arxiv.org/abs/1706.03762>), чтобы а) ввести параллелизм и б) позволить моделям обучаться длинным зависимостям в данных. Статья не только привела к появлению известных моделей в области обработки естественного языка, таких как BERT и GPT , но и к взрывному интересу применения трансформеров в различных областях, включая зрение. Хотя здесь мы рассмотрим модель для формирования подписей к изображениям, сама идея учета внимания при обучении нейросетей гораздо более общая.

Трансформер: Многоголовое внимание (multi-headed attention)

Внимание как скалярное произведение

Напомним, что внимание может рассматриваться как операция над запросом $q \in \mathbb{R}^d$, множеством векторов значений $\{v_1, \dots, v_n\}$, $v_i \in \mathbb{R}^d$, и множеством векторов ключей $\{k_1, \dots, k_n\}$, $k_i \in \mathbb{R}^d$, которая определяется выражением:

$$c = \sum_{i=1}^n v_i \alpha_i; \quad \alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)}$$

где α_i обычно называют "весами внимания", а результат операции $c \in \mathbb{R}^d$ представляет взвешенное среднее векторов значений.

Самовнимание

В Трансформерах выполняется операция самовнимания, которая означает, что значения (values), ключи (keys) и запросы (query) формируются на основе входных данных $X \in \mathbb{R}^{\ell \times d}$, где ℓ длина последовательности. При этом мы обучаем матрицы параметров $V, K, Q \in \mathbb{R}^{d \times d}$, чтобы обеспечить отображение входа X соответственно в векторы значений, ключей и запросов:

$$\begin{aligned} v_i &= Vx_i \quad i \in \{1, \dots, \ell\} \\ k_i &= Kx_i \quad i \in \{1, \dots, \ell\} \\ q_i &= Qx_i \quad i \in \{1, \dots, \ell\} \end{aligned}$$

Масштабированное многоголовое внимание в виде скалярного произведения

В случае многоголового внимания мы обучаем матрицы параметров для каждой головы, это позволяет модели лучше акцентировать внимание на различных частях входных данных. Пусть h - число голов, и Y_i выходное внимание головы i . Здесь мы обучаем отдельные матрицы Q_i, K_i и V_i . Для того, чтобы сохранить схему вычислений, такой же как и для одно-голового внимания, мы выбираем $Q_i \in \mathbb{R}^{d \times d/h}$, $K_i \in \mathbb{R}^{d \times d/h}$ and $V_i \in \mathbb{R}^{d \times d/h}$. Складывая и масштабируя $\frac{1}{\sqrt{d/h}}$ скалярное произведение в нашей простой модели внимания, получим

$$Y_i = \text{softmax}\left(\frac{(XQ_i)(XK_i)^\top}{\sqrt{d/h}}\right)(XV_i)$$

где $Y_i \in \mathbb{R}^{\ell \times d/h}$ и ℓ - длина последовательности.

В нашей реализации мы применяем регуляризацию dropout к весам внимания (хотя на практике её можно использовать на любом этапе):

$$Y_i = \text{dropout}\left(\text{softmax}\left(\frac{(XQ_i)(XK_i)^\top}{\sqrt{d/h}}\right)\right)(XV_i)$$

Окончательно, выход модуля самовнимания является линейным преобразованием A объединенных выходов голов Y_i :

$$Y = [Y_1; \dots; Y_h]A$$

где $A \in \mathbb{R}^{d \times d}$ и $[Y_1; \dots; Y_h] \in \mathbb{R}^{\ell \times d}$.

Реализуйте многоголовое внимание на основе масштабированного скалярного произведения в классе `MultiHeadAttention` в файле `dlcv/transformer_layers.py`. Код ниже обеспечит проверку вашей реализации. Относительная ошибка должна быть менее $e-3$.

Ввод [4]:

```

1 torch.manual_seed(231)
2 # Выбирайте измерения так, чтобы они все были уникальными для упрощения
3 # В частности, ниже выбраны следующие значения N=1, H=2, T=3, D//H=4 и
4
5 batch_size = 1
6 sequence_length = 3
7 embed_dim = 8
8 attn = MultiHeadAttention(embed_dim, num_heads=2)
9
10 # Самовнимание
11 data = torch.randn(batch_size, sequence_length, embed_dim)
12 self_attn_output = attn(query=data, key=data, value=data)
13
14 # Маскированное самовнимание
15 mask = torch.randn(sequence_length, sequence_length) < 0.5
16 masked_self_attn_output = attn(query=data, key=data, value=data, attn_m
17
18 # Внимание на основе 2-х разных входов
19 other_data = torch.randn(batch_size, sequence_length, embed_dim)
20 attn_output = attn(query=data, key=other_data, value=other_data)
21
22 # ожидаемые значения самовнимания
23 expected_self_attn_output = np.asarray([[
24 [-0.2494,  0.1396,  0.4323, -0.2411, -0.1547,  0.2329, -0.1936,
25          -0.1444],
26          [-0.1997,  0.1746,  0.7377, -0.3549, -0.2657,  0.2693, -0.2541,
27          -0.2476],
28          [-0.0625,  0.1503,  0.7572, -0.3974, -0.1681,  0.2168, -0.2478,
29          -0.3038]]])
30
31 # ожидаемые значения маскированного самовнимания
32 expected_masked_self_attn_output = np.asarray([[
33 [-0.1347,  0.1934,  0.8628, -0.4903, -0.2614,  0.2798, -0.2586,
34          -0.3019],
35          [-0.1013,  0.3111,  0.5783, -0.3248, -0.3842,  0.1482, -0.3628,
36          -0.1496],
37          [-0.2071,  0.1669,  0.7097, -0.3152, -0.3136,  0.2520, -0.2774,
38          -0.2208]]])
39
40 # ожидаемые значения внимания на основе 2-х разных входов
41 expected_attn_output = np.asarray([[
42 [-0.1980,  0.4083,  0.1968, -0.3477,  0.0321,  0.4258, -0.8972,
43          -0.2744],
44          [-0.1603,  0.4155,  0.2295, -0.3485, -0.0341,  0.3929, -0.8248,
45          -0.2767],
46          [-0.0908,  0.4113,  0.3017, -0.3539, -0.1020,  0.3784, -0.7189,
47          -0.2912]]])
48
49 print('ошибка самовнимания: ', rel_error(expected_self_attn_output, sel
50 print('ошибка маскированного самовнимания: ', rel_error(expected_masked
51 print('ошибка внимания: ', rel_error(expected_attn_output, attn_output.

```

ошибка самовнимания: 0.0003775124598178026

ошибка маскированного самовнимания: 0.00015258647520338531

ошибка внимания: 0.00035202796492688154

Позиционное кодирование

Хотя трансформеры способны акцентировать внимание на любой части входных данных, механизм внимания не имеет понятия о порядке следования токенов. Для многих задач (особенно обработки естественного языка) относительный порядок токенов очень важен. Чтобы обеспечить учет порядка следования токенов, в схему трансформера добавляют позиционное кодирование при встраивании отдельных токенов.

Определим матрицу $P \in \mathbb{R}^{l \times d}$, где $P_{ij} =$

$$\begin{cases} \sin\left(i \cdot 10000^{-\frac{j}{d}}\right) & \text{если } j \text{ четное} \\ \cos\left(i \cdot 10000^{-\frac{(j-1)}{d}}\right) & \text{иначе} \end{cases}$$

Вместо того, чтобы напрямую подавать на вход нейросети $X \in \mathbb{R}^{l \times d}$, на её вход подают $X + P$.

Реализуйте этот слой в классе `PositionalEncoding` в файле `dlcv/transformer_layers.py`. С помощью кода ниже протестируйте вашу реализацию. Вы должны получить ошибку порядка $e-3$ или менее.

Ввод [5]:

```
1 torch.manual_seed(231)
2
3 batch_size = 1
4 sequence_length = 2
5 embed_dim = 6
6 data = torch.randn(batch_size, sequence_length, embed_dim)
7
8 pos_encoder = PositionalEncoding(embed_dim)
9 output = pos_encoder(data)
10
11 expected_pe_output = np.asarray([[[-1.2340,  1.1127,  1.6978, -0.0865,
12                                     [ 0.9028, -0.4781,  0.5535,  0.8133,
```

ошибка позиционного кодирования: 0.00010421011374914356

Вопрос 1

Выше мы представили несколько ключевых решений, которые были положены в основу при проектировании масштабированного внимания в виде скалярного произведения. Объясните, почему следующие решения были выгодны:

1. Использование нескольких голов внимания вместо одной.
2. Деление на $\sqrt{d/h}$ перед применением функции `softmax`. Вспомните, что d — это размерность признака, а h — это количество голов внимания.
3. Применение линейного преобразования к выходным данным операции внимания.

Необходимо дать краткий ответ по каждому пункту, но обязательно будьте конкретны в описании того, что произошло бы без каждой данной детали реализации, почему такая ситуация была бы неоптимальной и как предлагаемая реализация улучшает ситуацию.

Ваш ответ:

1. Использование нескольких голов внимания вместо одной. Это позволяет модели одновременно фокусироваться на разных аспектах или подпространствах входных данных (например, на синтаксисе и семантике). Без этого (с одной головой) модель была бы ограничена одним типом отношений между словами, что снизило бы её выразительную способность и качество предсказаний.
2. Деление на $\sqrt{d/h}$ перед softmax. Это масштабирование предотвращает чрезмерно малые градиенты в функции softmax, которые возникают при больших значениях скалярного произведения (дисперсия растёт с ростом размерности). Без этого масштабирования градиенты становились бы слишком маленькими, что замедляло или останавливало обучение, а данная нормализация стабилизирует процесс.
3. Применение линейного преобразования к выходным данным операции внимания. Это позволяет модели смешивать и перераспределять информацию, полученную от разных голов внимания, в единое представление нужной размерности. Без этого преобразования выход был бы просто конкатенацией, что не давало бы сети возможности обучаться оптимальному взвешиванию признаков, снижая гибкость и качество конечного представления.

Трансформер для подписи изображений

Теперь, когда вы реализовали слои, описанные выше, вы можете объединить их для создания модели, обеспечивающей формирование подписей к изображениям с использованием трансформера. Откройте файл `dlcv/classifiers/transformer.py` и просмотрите класс `CaptioningTransformer`.

Реализуйте в классе функцию `forward`. После этого выполните проверку прямого распространения используя небольшой тест ниже. Вы должна получить относительную ошибку $e-5$ или менее.

Ввод [6]:

```

1 torch.manual_seed(231)
2 np.random.seed(231)
3
4 N, D, W = 4, 20, 30
5 word_to_idx = {'<NULL>': 0, 'cat': 2, 'dog': 3}
6 V = len(word_to_idx)
7 T = 3
8
9 transformer = CaptioningTransformer(
10     word_to_idx,
11     input_dim=D,
12     wordvec_dim=W,
13     num_heads=2,
14     num_layers=2,
15     max_length=30
16 )
17
18 # Выбираем фиксированные значения для всех параметров модели
19 for p in transformer.parameters():
20     p.data = torch.tensor(np.linspace(-1.4, 1.3, num=p.numel()).reshape
21
22 features = torch.tensor(np.linspace(-1.5, 0.3, num=(N * D)).reshape(N,
23 captions = torch.tensor((np.arange(N * T) % V).reshape(N, T))
24
25 scores = transformer(features, captions)
26
27 expected_scores = np.asarray([[[-16.9532, 4.8261, 26.6054],
28     [-17.1033, 4.6906, 26.4844],
29     [-15.0708, 4.1108, 23.2924]],
30     [[-17.1767, 4.5897, 26.3562],
31     [-15.6017, 4.8693, 25.3403],
32     [-15.1028, 4.6905, 24.4839]],
33     [[-17.2172, 4.7701, 26.7574],
34     [-16.6755, 4.8500, 26.3754],
35     [-17.2172, 4.7701, 26.7574]],
36     [[-16.3669, 4.1602, 24.6872],
37     [-16.7897, 4.3467, 25.4831],
38     [-17.0103, 4.7775, 26.5652]]])
39

```

ошибка scores: 5.056720614342753e-06

Переобучение модели трансформера на небольшом объеме данных

Выполните переобучение модели трансформера, формирующего подписи к изображениям, на том же небольшом наборе данных. В этом задании используется класс `CaptioningSolverTransformer` для обучения моделей, формирующих подписи к изображениям.

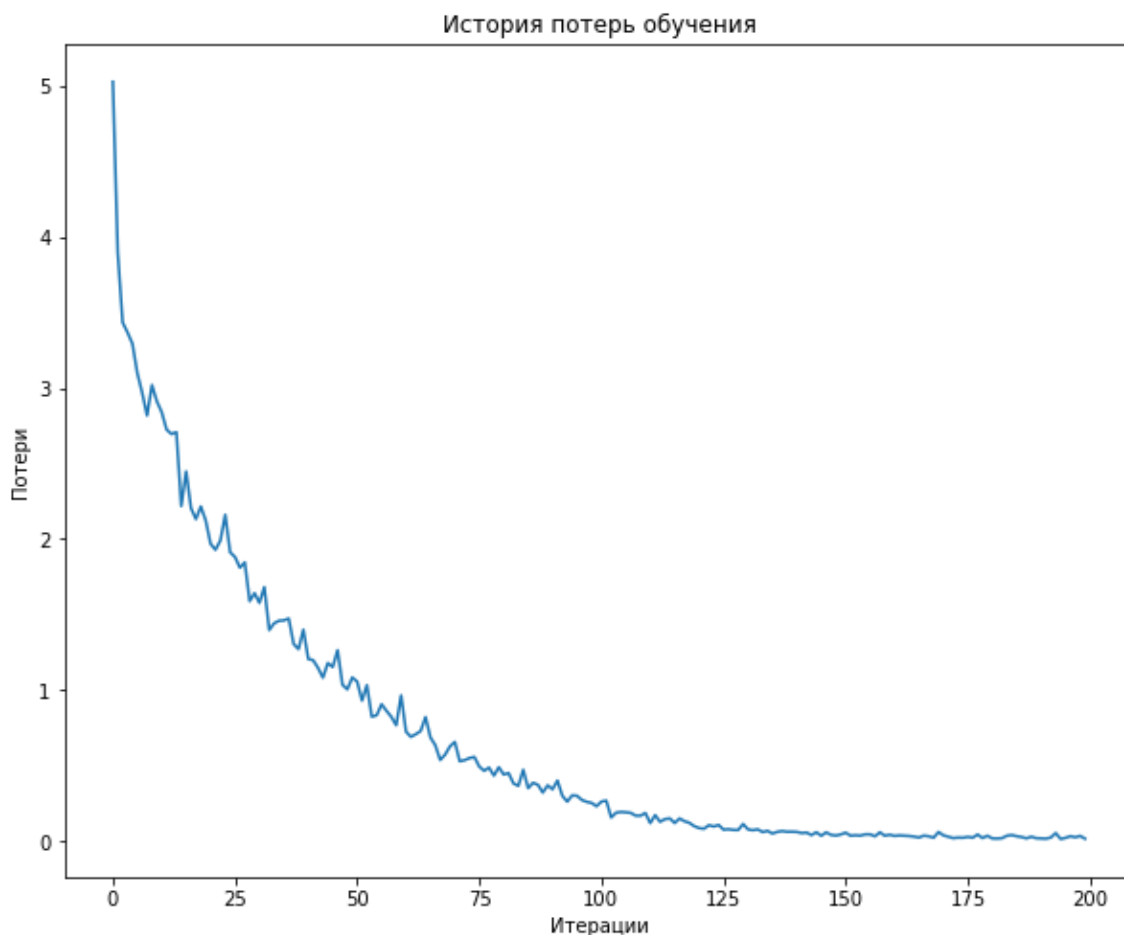
Выполните ячейку ниже, чтобы убедиться, что модель переобучается на небольшом наборе из 100 обучающих примеров.

Ввод [10]:

```
1 torch.manual_seed(231)
2 np.random.seed(231)
3
4 # загрузка данных (50 обучающих примеров)
5 data = load_coco_data(max_train=50)
6
7 # создание Трансформера для формирования заголовков
8 transformer = CaptioningTransformer(
9     word_to_idx=data['word_to_idx'],
10    input_dim=data['train_features'].shape[1],
11    wordvec_dim=256,
12    num_heads=2,
13    num_layers=2,
14    max_length=30
15 )
16
17 # создание экземпляра класса для обучения Трансформера
18 transformer_solver = CaptioningSolverTransformer(transformer, data, idx
19    num_epochs=100,
20    batch_size=25,
21    learning_rate=0.001,
22    verbose=True, print_every=10,
23 )
24 # обучение трансформера
25 transformer_solver.train()
26
27 # Построение графика потерь
28 plt.plot(transformer_solver.loss_history)
29 plt.xlabel('Итерации')
30 plt.ylabel('Потери')
31 plt.title('История потерь обучения')
```

```
base dir C:\Users\HP\DL_lab\2026_transformer_ решение\dllcv\datasets/coco_
captioning
```

```
(Iteration 1 / 200) loss: 5.023861
(Iteration 11 / 200) loss: 2.838942
(Iteration 21 / 200) loss: 1.969251
(Iteration 31 / 200) loss: 1.578352
(Iteration 41 / 200) loss: 1.207504
(Iteration 51 / 200) loss: 1.058034
(Iteration 61 / 200) loss: 0.727213
(Iteration 71 / 200) loss: 0.659099
(Iteration 81 / 200) loss: 0.443693
(Iteration 91 / 200) loss: 0.345755
(Iteration 101 / 200) loss: 0.264495
(Iteration 111 / 200) loss: 0.122238
(Iteration 121 / 200) loss: 0.089583
(Iteration 131 / 200) loss: 0.081036
(Iteration 141 / 200) loss: 0.064176
(Iteration 151 / 200) loss: 0.059066
(Iteration 161 / 200) loss: 0.037873
(Iteration 171 / 200) loss: 0.041521
(Iteration 181 / 200) loss: 0.021156
(Iteration 191 / 200) loss: 0.021602
```



Печать финальных потерь обучения. Вы должны получить финальные потери обучения меньше, чем 0.03.

Ввод [11]:

```
Финальные потери: 0.019136833
```

Тестирование трансформера

Модели, формирующие подписи ведут себя по-разному во время обучения и во время тестирования. Во время обучения у нас есть доступ к истинному заголовку, поэтому мы вводим истинные слова в качестве выходных данных модели на каждом этапе. Во время тестирования мы предсказываем слово на отдельном временном шаге и подаем это слово на вход модели на следующем временном шаге.

Выполните ячейку ниже, чтобы получить заголовки на выходе переобученной модели как для обучающих изображений, так и для проверочных. Подписи для обучающих изображений должны быть очень хорошими; для проверочных изображений они, вероятно, не будут иметь смысла.

Ввод [15]:

```

1 # Если вы получили ошибку "URL не найден" не волнуйтесь!
2 # Вы можете повторно выполнить ячейку столько раз, сколько захотите.
3 for split in ['train', 'val']:
4     minibatch = sample_coco_minibatch(data, split=split, batch_size=2)
5     gt_captions, features, urls = minibatch
6     gt_captions = decode_captions(gt_captions, data['idx_to_word'])
7
8     sample_captions = transformer.sample(features, max_length=30)
9     sample_captions = decode_captions(sample_captions, data['idx_to_wor
10
11     for gt_caption, sample_caption, url in zip(gt_captions, sample_capt
12         img = image_from_url(url)
13         # Пропустить отсутствующий URLs.
14         if img is None: continue
15         plt.imshow(img)
16         plt.title('%s\n%s\nGT:%s' % (split, sample_caption, gt_caption))
17         plt.axis('off')
18

```

train

a large truck parked on the cement in front of a building <END>
 GT:<START> a large truck parked on the cement in front of a building <END>

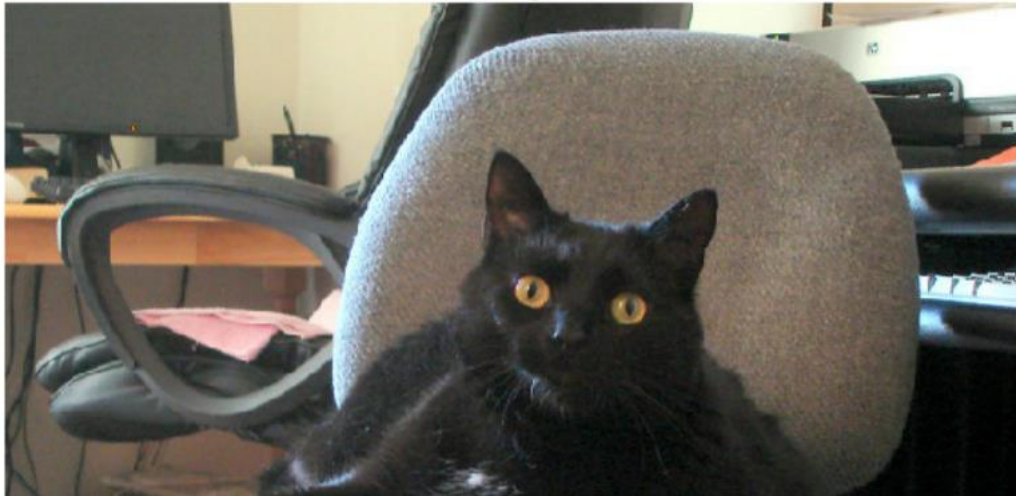


Ввод []:

train

a black cat sitting in an office chair <END>

GT:<START> a black cat sitting in an office chair <END>



val

a red truck that <UNK> and a school coming bus <END>

GT:<START> a city train stopped at the train station <END>



val

a dog the <UNK> <UNK> during a bed of a <UNK> <END>
GT:<START> two men playing tennis outside during the day <END>

